RevTrace: Preserving Valid Progress across Trusted Feedback Revisions

Anonymous submission
Regular paper

***Abstract*—Feedback-driven inference may withdraw a still-supported view even after timely delivery. RevTrace addresses these avoidable withdrawals by coordinating complete-input eligibility, original selection authority, and whole-view support. It retains eligible calls without endorsing their predictions and preserves matching received results across producer failure. We give sufficient retention conditions and a conditional selected-trace refinement argument. In a 24-speaker factorial study, computation retention increases frontier withdrawals from 12 to 17; joint retention yields zero in the preselected setting. In a separate native-correction study on eight previously unused speakers, dual-deadline audio coverage rises from 40.6% to 93.9% over conservative execution, with unchanged selected histories and final quality. The speaker-paired gain is 53.3 percentage points (95% CI [44.8, 59.6]); every source improves. Whole-run passes increase from 0/16 to 10/16. Controlled failures validate retention of matching received results and rejection of mismatches.**

***Index Terms*—dependable inference, feedback state, incremental computation, speculative execution, streaming ASR**

I. Introduction

A trusted revision can withdraw progress whose computation remains valid. In streaming speech correction, selected corrections feed later recognition [1]; trusted feedback means acceptance by the original protected selection process. We call the loss of a still-supported whole view an *avoidable withdrawal*. In Figure 1(a), display within two seconds (D2) and certified coverage within eight seconds (C8) both meet their deadlines, yet the view becomes unavailable for 2.176 s.

First delivery is a past event; a consumer reads the current view. Figure 1(a) exposes a service gap without a corresponding refutation; Figure 1(b) preserves the shown coverage but misses D2 by 27.4 ms. We therefore evaluate continued availability separately from first delivery.

Retaining computation alone can make delivered progress less stable. In a preselected 24-speaker Qwen setting with 2-s feedback and staggered arrivals, it increases *frontier withdrawals*—backward movements of the displayed audio frontier—from 12 to 17, despite improving timely audio coverage. Joint computation and view retention yields zero in that setting. The difference arises at the Consumer: installing a compatible 12-s state can overwrite a supported view already covering 14 s. RevTrace preserves that usable progress while keeping the original selection task unchanged.

The challenge is to act on evidence that arrives at different times. A revision can remove a view's support without refuting the unresolved input of a future call; confirming an input does not endorse the call's own predicted successor. Thus one revision can require different actions for an executing call, an accepted result and a displayed hypothesis. RevTrace therefore checks complete-call input eligibility [EL], original content-selection authority [AU], and whole-view support [SUP] separately (Figure 2). Producer exit can end new-message admission while an already accepted result remains eligible. A birth version, matching text, or producer-liveness bit cannot stand in for these decisions.

Dependency reuse and revocation have established foundations in incremental dialogue and computation [2]–[4]. SSBD coordinates draft reuse with display masking [5]; PredGen speculates across evolving speech stages [6]. RevTrace realizes coordination around a recursively selected feedback task: complete calls retain the original correction and update, while the actual whole view is checked independently at selected-state installation, binding reuse to the unchanged recursive task.

We make three contributions:

- A service contract exposing avoidable withdrawal and sufficient, executable witnesses for complete-call eligibility and whole-view support across trusted revisions (Sections II–III).
- A joint coordinator that preserves the original selected transition relation while retaining eligible calls and supported whole views. A selected-trace argument, bounded normal/failure-state exploration, and receipt-based checks of real executions support its conditional guarantees (Sections III–IV).
- Evidence that computation retention alone can worsen delivered progress, and that confirmed full calls improve timely service while preserving the selected task. Across eight previously unused native speakers, coverage increases by 53.3 percentage points (paired-speaker 95% CI [44.8, 59.6]), with positive paired gains for all eight sources (Section V).

II. System and Feedback Contract

A. System and Failure Model

Each session has an Owner serializing selected transitions, workers executing frozen calls, and an actual Consumer applying selected facts and complete provisional hypotheses. Durable storage records the selected prefix. A Supervisor monitors the processes and preserves the accepted-input ledger.

We assume one trusted Owner per session, protected selection, immutable audio and model configuration, unambiguous encoding, collision-resistant digests, and storage honoring

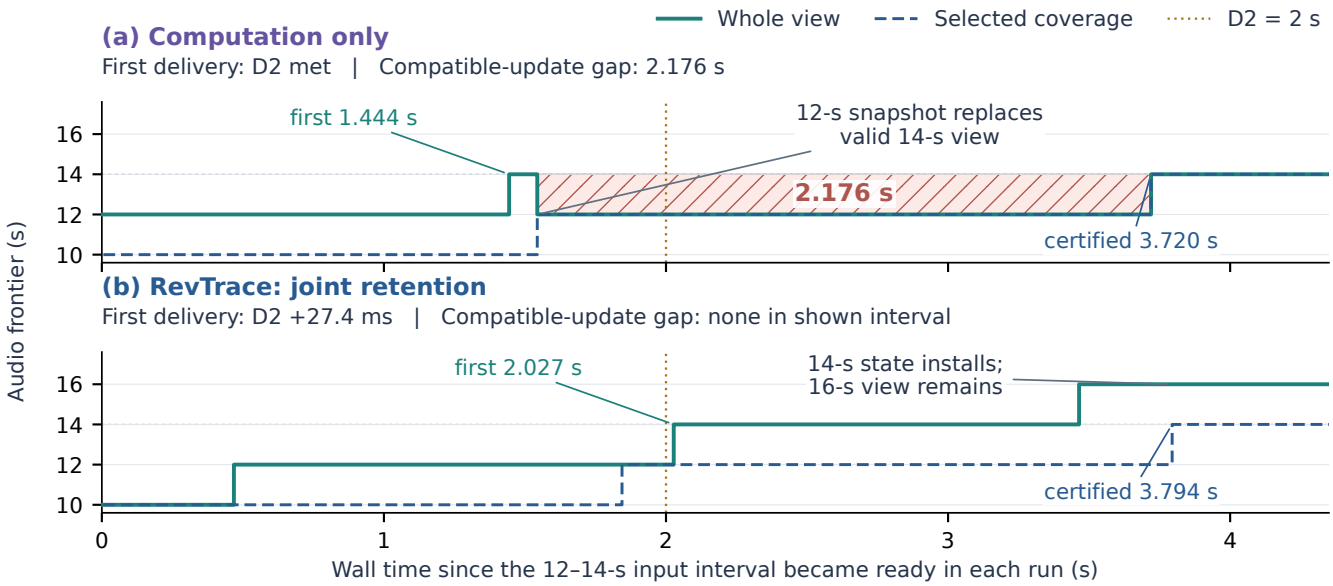


Fig. 1. On-time delivery does not guarantee continued availability. Two actual c16-overload runs process the same complete 12–14-s audio interval on their own ready clocks. (a) Computation-only delivers 37 new code points within D2, then loses the supported view for 2.176 s before identical text returns. (b) RevTrace misses D2 by 27.4 ms but preserves coverage when its 14-s state installs beneath a supported 16-s view. Both certified deliveries meet C8 (8 s). The executions have different interleavings; they are not an event-matched counterfactual.

successful synchronization. Calls admitted for use must execute their frozen inputs faithfully. Host receipts bind API objects, not GPU arithmetic; independent Private and Authority floating-point executions need not agree.

The interface checks identity, completeness, derived state, and ancestry. Native resilience supports worker or Owner-group fail-stop on an intact filesystem; Supervisor, accepted-input ledger, actual Consumer, power observer, and synchronized storage survive. Accepted inputs retain their original arrival clocks through restart. Supervisor/Consumer failure, host power loss, compromised Owner or Authority, Byzantine code, persistent corruption, and lost storage are excluded. Controlled wrong-input injections in Section V intentionally violate normal Private input fidelity; we use them to study containment and executor release, not as examples covered by the lawful-continuation proposition.

B. Feedback creates an execution dependency

The service has two logical paths: protected selection establishes *Canonical* facts, while a *Private* path computes and displays provisional hypotheses. Feedback links successive calls: StreamCorrect feeds selected corrections into later ASR [1]. We instantiate these roles in two execution settings. The core runtime uses separate Private and Authority workers; the native runtime uses two complete recognizer–corrector workers, either able to execute a selected-input or conditional full call. Both coordinate computation with the feedback that the original program eventually selects.

For a fixed session and execution contract, write an installed fact as

$$C = (v, F, Z_C, P_C, T_C). \tag{1}$$

Here v is the version, F the certified processing frontier, Z_C the complete selected output, P_C the feedback state, and T_C its bound text. The contract fixes the model, adapter, decoding, audio identity, and transformations to P and T. Consumer coverage follows the adapter's publication policy; advancing F need not add characters. Certification means acceptance by protected selection and installation, separately from human-transcription accuracy.

The adapter declares a conservative representation of *all mutable cross-call state that can affect later model inputs, selection, or feedback*. Telemetry and resource accounting are recorded separately from this inference-relevant state; any configuration affecting inference remains bound by the contract. Qwen passes prompt tokens without cross-call numerical KV caches. The native adapter additionally carries audio buffers, token groups, context, alignment position, and decoder configuration. Comparisons require the same contract: text equality cannot replace an omitted cursor or numerical cache. The representation may conservatively include fields unnecessary to a particular call.

C. Calls, views and outstanding input

A complete provisional node n records a frozen call

$$q_n = (a_n, P_n^-, b_n, \chi_n), \quad n = (q_n, Z_n, P_n^+, T_n), \tag{2}$$

where a_n is the input-state frontier, $b_n > a_n$ the audio target, and χ_n binds remaining inputs and execution identity. The model may read the prefix through b_n; a_n does not assert token-to-audio alignment. The adapter derives P_n^+ and T_n from a complete result. An in-flight call has only its frozen input; partial or cancelled tokens supply no node.

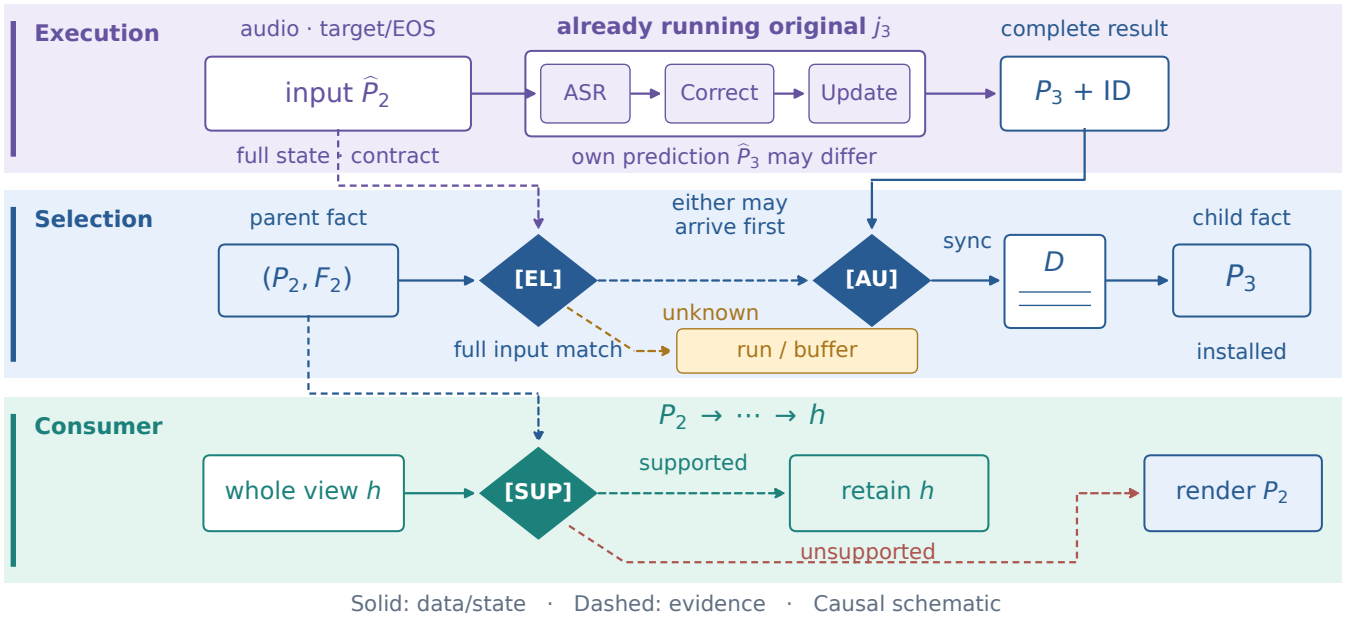


Fig. 2. Early execution and later confirmation meet at selection. The input packet binds audio, target/EOS, full state and model/decoder configuration. Original call j_3 already runs; its result may complete before or after [EL] confirms the actual parent. [AU] joins that complete result, confirmed current parent and owned identity; synchronizing fact D precedes installation. [SUP] independently checks the whole visible view. The Consumer lane shows application of the parent fact P_2; the child fact P_3 triggers its own [SUP] check when applied. Pending input permits running/buffering without commit. Solid arrows carry data/state; dashed arrows carry check evidence. This is a causal schematic, without a measured time scale.

Computation, display and resource use need not change together. An update may remove support for a displayed hypothesis while its producer still runs; an accepted result may remain usable after that producer exits. RevTrace therefore tracks dependency validity, whole-view support and physical completion separately.

The service retains every accepted input obligation until certified processing covers it. Discarding a branch, requesting cancellation, or recovering an old view does not remove that obligation or reset its arrival clock. In exact mode, a missed deadline never authorizes an uncertified Canonical transition.

III. Coordinating Calls and Whole Views

A. One revision, three different decisions

Figure 2 follows call j_3, started on provisional $\widehat{P}_2$ before its parent's original Correct/Update selects P_2. A full-input match at P_2 confirms [EL], preserving j_3's identity. Its own prediction $\widehat{P}_3$ may still differ from the original selected result P_3: [AU] selects that complete result only after its actual parent is installed. Independently, installing P_2 checks [SUP] on the current whole view h. Losing h's support does not refute a later call's unknown input. Table I separates the three objects and uses. Subscripts denote states; each call freezes all contract inputs. In the evaluation, R/S switch retention actions under these judgments.

B. Complete dependency witnesses

A birth version records dispatch history; the dependency witness establishes whether a call remains usable under the installed state.

TABLE I
Three judgments govern different uses of speculative work.

Object	Permits	Still requires
[EL] Full call input	Confirmed: retain Unknown: run/buffer Refuted: reject use	[AU] for selection; [SUP] for a view.
[AU] Original result and parent	Durable selection after full-input confirmation.	[SUP] for the actual ahead view.
[SUP] Actual whole view and path	Retain at an authorized compatible application.	[EL] for calls; [AU] for selected transitions.

Let $x = (F, P_C)$. A complete recorded node provides an edge

$$(F, P_C) \xrightarrow{n} (b_n, P_n^+) \quad \text{if} \quad a_n = F \ \wedge \ P_n^- = P_C, \tag{3}$$

under the input, identity, and execution contract of Section II. A finite sequence of such edges is a *valid provisional continuation* of C when each successor consumes the preceding complete output state at its actual frontier. Write $C \rightsquigarrow n$ for a complete path to n. A frozen in-flight successor can belong to the same continuation, but contributes no usable result until completion.

Proposition 1 (Sufficient re-rooting witness)**.** *Suppose an old branch contains a complete boundary node at the new certified frontier F, with output state P_C. If its subsequent recorded calls form an unbroken input–output path to the actual branch tip, those nodes are a valid provisional continuation of C. An in-flight successor may continue when its unchanged frozen input is that tip and its target exceeds F.*

Proof. The boundary equals (F, P_C), so the first successor satisfies Eq. (3). Each subsequent call consumes the preceding complete output at its recorded frontier under the same contract. Induction reaches the tip and the unchanged input of its in-flight successor, which supplies no output yet. □

This establishes dependency validity, not Canonical write authority; provisional results remain revisable.

A branch born under v can reach (F, P_C) when $v+1$ installs, making version-only rejection wasteful. Conversely, equal text can hide different tokens or alignment state; equal state at a different frontier also fails Eq. (3).

C. [EL] Retaining computation: R

The core Qwen guard conservatively implements Proposition 1. On installation, it requires a complete Private node at exactly F with full feedback tokens P_C, then checks frozen inputs and frontiers along complete successors to the actual tip. An intermediate match is insufficient. An in-flight successor must be the unique tracked task, consume that tip, and target audio beyond F.

Success retains the branch without changing job IDs, birth versions, inputs, or results; otherwise the Owner fences it and rebuilds from an allowed state. Boundary-node and unique-slot requirements restrict this sufficient guard, not every valid reuse scheme.

D. [SUP] Retaining an actual view: S

A snapshot Consumer can replace a valid ahead view with a shorter certified snapshot despite computation retention. RevTrace maintains separate certified and provisional whole-hypothesis lanes, without an uncertified suffix splice or certified-prefix status for retained views.

For a Canonical application, S checks the complete visible node's input directly against (F, P_C) or follows complete published ancestors to that boundary. Missing, cyclic, or ambiguous ancestry is rejected. Unlike R's conservative guard, a directly rooted published call needs no extra Private boundary node.

Before actual application and its receipt, the kernel checks the preview against the durable root, registered results, dependency path, and bound text/frontier. A caller-supplied compatibility flag cannot replace these records.

Proposition 2 (Continuity at a compatible application)**.** *Immediately before an authorized Canonical application, let the visible whole provisional view h be ahead of its new frontier F. If S proves $C \rightsquigarrow h$ and no independent withdrawal mode is invoked, that application advances the certified lane and preserves h unchanged.*

Proof. The authorized compatible-and-ahead branch installs C in the certified lane while retaining h's node, text, and frontier. That application therefore withdraws none of h. □

Incompatible updates must withdraw the view, and later valid results can revise it. This is a per-application property, not global monotonicity. Snapshot consumption and conservative recomputation can be safe; the four-arm study measures the extra progress and cost of joint retention.

E. Native feedback and conditional full calls

StreamCorrect supplies a concrete recursive feedback task [1]. For complete closed state P and accepted audio X, expose its recognition boundary as

$$A(P, X) = (B, M), \tag{4}$$

where B contains ordered beam candidates and M the engine state and inputs needed by original Correct/Update. Candidate order, prompts, returned token IDs, selection, and update rules remain intact.

Before learned correction finishes, the original update tail with a typed first-beam identity prediction produces provisional closed state $\widehat{P}^{+}$. It is a dependency projection, not a complete selected result. A second worker can run the next *complete original call* on that conditional input; both GPUs host the full recognizer–corrector. Its own prediction may be wrong even when its input later matches. Original Correct/Update determines the complete result.

When the parent installs, the Owner compares the call's complete frozen start state, actual frontier, contract, and session epoch with it. Equality confirms the unchanged owned call: it continues if running, or becomes eligible for ordered selection if complete. Job ID, birth, producer incarnation, inputs, and execution history stay unchanged.

An earlier ancestor mismatch can remove current view support without refuting a later complete input. Cancellation before correction requires a disagreeing actual selected state at that call's own start frontier, coverage of its target without remaining use, or a changed epoch. An unresolved future parent supplies none of these proofs. The view can therefore require withdrawal while the call remains unresolved.

This native input witness and the core Qwen boundary-node/unique-successor guard are different sufficient implementations of the continuation relation. Neither grants write authority. Both use a whole-view Consumer that keeps certified and provisional provenance separate. The native comparison holds two complete workers fixed and varies conditional dispatch and retention; ready-only ASR handoff is a separate developmental control. For a call j, its complete frozen input is *confirmed* when it matches the installed parent, *refuted* when an actual selected state at its start frontier differs, and *unknown* without either witness. Covered targets and fenced sessions cannot supply selected results. Table II separates these input decisions from actual view application.

The dispatcher scans available workers and sessions in their existing order. Within each session it tries missing selected work before a conditional tail, subject to the ahead bound; it imposes no global selected-work priority across sessions. Unknown input grants neither selection nor a refutation-based cancellation proof. Replacing a typed prediction with its complete result keeps only the suffix that consumes the actual output contiguously. Physical release and producer-failure handling follow Section IV.

TABLE II
EVENT-DRIVEN COORDINATION OF CONFIRMED CALLS AND SUPPORTED WHOLE VIEWS.

Event	Required evidence	Action
Dispatch	Ready worker; accepted audio and an eligible session start	Freeze the full input, target and original job identity.
ASR result	Bound recognition result and contiguous provisional path	Extend the typed prediction and publish its provisional view.
Full result	Bound original result [AU]; complete input status [EL]	Select if confirmed; buffer if unknown. Reject refuted/covered results and fenced messages.
Select ready	Confirmed complete result and unchanged installed parent	Synchronize the bound fact, install, and recheck pending calls.
Consumer update	Ordered fact, actual parent and whole-view support [SUP] $S(C, h)$	Advance the certified view. Retain the complete ahead h when supported; otherwise render the installed state.

F. [AU] Preserving the selected program

State-machine accounts relate service execution to ordered commands [7]. Let $P \Rightarrow_{\mathcal{A},X} P'$ denote a transition allowed by the original protected program on accepted immutable input X. The relation may be nondeterministic. A complete original call includes its original selection and update, or its declared buffer-only path; a typed identity projection is not such a result. We project a protocol execution onto its distinct durable selected facts, deleting speculation, waiting, local rejection, cancellation, collection, and repeated delivery. This restricted projection follows the standard stuttering view of refinement [8]; provisional whole-view behavior remains outside its alphabet and has a separate compatibility property.

Early execution must not create an additional selected transition merely because a prediction was available. The following condition binds every selected advancement to its original complete result.

Proposition 3 (Selected-trace refinement)**.** *Assume sufficient declared state, faithful execution of frozen inputs, exclusive serial selection, correctly bound message identities, and storage that honors successful synchronization. If each new durable fact binds a complete original call whose full input equals the preceding selected cut, retains the call's original identity, was admitted under valid message authority, and is synchronized before installation or certified application, then the projected facts form an execution allowed by $\mathcal{A}$. Replaying an existing fact adds no transition.*

Proof. The initial cut belongs to $\mathcal{A}$. Deleted actions do not extend the prefix. An early completion remains buffered while its parent input is unknown. Exclusive serial selection revalidates its actual parent and complete frozen input before append, so completion order adds no parentless edge. Each new fact binds a complete original call on exactly the preceding cut and accepted input, hence an allowed next edge regardless of dispatch time. Synchronization precedes privileged application. Recovery replays the same checked identities, including a fact first consumed after an Owner crash, without adding an edge. Induction proves the claim. □

Propositions 3 and 2 govern original selection [AU] and compatible whole-view retention [SUP], respectively. Input confirmation [EL] retains work without endorsing its prediction. Meeting deadlines further depends on when views and complete original results reach the actual Consumer.

a) Specification checks and execution evidence: A separately frozen, hand-written specification explores 6,231 two-frontier normal states and 14,074 one-frontier failure states, each to a 12-event depth, with 27 named controls. An independent trace checker binds actual facts and whole-view applications to original call receipts. These bounded checks complement the conditional argument; they are not a mechanically extracted whole-program proof.

IV. EXECUTION PROTOCOL

A. Complete results and physical ownership

In the core runtime, dispatch binds session, incarnation, job, input, target, and contract. A row returns either a complete result with re-derived state/text or cancellation without partial state. Complete rows can be installed and consumed before their batch finishes. Only all terminal rows and device synchronization authorize `BatchDrained` and executor reuse. The native process lifecycle below has separate evidence.

B. Covered work and safe-point exit: K

For the tested use model, K requests exit when the installed Canonical frontier covers a proposal's target and it has no remaining declared use. This differs from R: valid work beyond the new root can remain useful. Workers observe `CancelRequest` at decode boundaries; completion can win, but each row has one terminal outcome. Static batches are not compacted or refilled; masked costs and drain delays remain measured.

C. Durable facts and outstanding input

Before acknowledgment, synchronized frames bind parent/next states, request identity, selected complete result, and receipt; recorded deltas must reconstruct the state. Group-ready commit shares synchronization among validated sessions without a collection timeout or cross-session atomic transaction. Uncertain append/application outcomes block writes. Write-ahead recovery [9] reconstructs facts and identities; the native extension binds this to original correction calls.

D. Continuing through delivery and process failures

Each job has an Owner namespace and monotonic issue number; each physical slot has a worker incarnation, using established RPC identity principles [10]. Fencing ends authority to send new evidence while preserving the identity of an already accepted complete result.

a) Local rejection and physical release: Exact duplicates and fenced messages are rejected locally; they cannot change selected state or release newer leases. Contradictory currently owned contents freeze writes. Per-incarnation Pipes and receivers isolate incomplete frames. Replacement requires process exit, CUDA release and joined receivers. Complete receipts from the sealed channel are validated before fencing. A retained result still needs its original full input to match the selected parent; producer exit supplies no such evidence.

b) Durable selection and recovery: A synchronous hook binds complete parent/next states, original correction and IDs, job, Owner/session epochs, and producer incarnation. Framing, checksums and chaining protect the fact; synchronization precedes installation/publication. Uncertain writes freeze the writer. Recovery discards only an incomplete final frame; complete corruption fails. These conventional write-ahead rules [9] bind the original feedback transition.

The surviving Supervisor preserves the input ledger and original clocks. After old physical ownership ends, it advances the Owner namespace, replays the verified fact prefix, and cold-starts workers. Replay adds no inference; old publications gain no new authority. Conservative execution and RevTrace share this coordinator, durability, recovery and actual Consumer.

c) Conditional continued service: For finite accepted work after finite supported failures, further progress requires readable storage, eventual physical release/loading, terminating original calls that advance frontiers, fair Owner/Consumer scheduling, and sufficient service budgets. Recovery keeps the original input clocks: loading, replay and recomputation all count toward the deadlines. These progress conditions establish no fixed recovery-time or D2/C8 bound.

d) Control history: Calls, buffers, views and outboxes pin their dependencies; acknowledgment releases only publication references. Namespace/issue high watermarks prevent reuse after collection. Unreferenced terminal/audit windows are bounded and diagnostics stream to disk, separately from active state, cumulative text and total memory.

V. Evaluation

A. Experimental design, comparisons and metrics

Three complementary studies test the coordination under unchanged selected tasks. Core factorial controls isolate whether retained computation becomes continuously available service [EL]/[SUP]. Native tests ask whether early complete calls preserve the original recursively selected task [EL]/[AU]. Process-failure tests distinguish already accepted results from future message authority. Their benefits arise under different execution contracts and are evaluated separately. An instrumented FIFO Consumer records actual applications. All accepted inputs keep their original ready times through cancellation and recovery.

a) Core implementation and sources: Two 24-GB RTX 3090s host an FP16 Private worker and FP32 Authority worker. Qwen3-ASR 1.7B [11] is primary; Whisper base.en [12] tests a second state adapter. Complete-row delivery, at most 16 rows per batch, a 256-token limit, the protocol kernel and synchronized journals are shared. Physical early exit is disabled in this factorial study, separating retention from cancellation. Initial loading precedes the service clock. Energy integrates both physical boards over the declared service window.

Independent confirmation uses 24 previously unused LibriSpeech speakers [13], metadata-selected before inference and partitioned into three eight-speaker groups (cohorts), totaling 714.110 s per arm. Its 110 cells cover primary controls, fresh-process repeats and coarser feedback schedules. At $\rho = 1$, Authority advances every 2 s; $\rho = 3$ changes selection frequency and feedback segmentation. Arrivals are simultaneous or staggered by 0.25 s. We reuse public recordings without collecting participant data; repetitions share speakers, the independent acoustic units.

b) Named comparisons: Checked snapshot + recomputation (R_0S_0), *View retention only* (R_0S_1), *Computation retention only* (R_1S_0), and *RevTrace: joint retention* (R_1S_1) share validation, durability, input accounting and physical execution. Disabling R rebuilds work from selected feedback; disabling S replaces the provisional view with the installed snapshot. These switches enable retention actions; all four arms share input-admission and view-application safety checks. The *alternative joint policy* is our positive control: another instantiation of the same retention and Consumer conditions on the shared execution/commit substrate. External prior work and our author-built comparators have separate identities.

c) Metrics: For offered interval i, w_i is audio duration, r_i its original ready time, d_i first actual visible coverage and c_i first actual certified coverage. Audio coverage meeting both deadlines is

$$A_{2,8} = \sum_i w_i \, \mathbf{1}\{d_i - r_i \le 2 \ \wedge \ c_i - r_i \le 8\}. \qquad (5)$$

All offered intervals, including partial end-of-stream (EOS) tails, remain in the denominator. Whole-run passes require display P99 at most 2 s (D2) and certified-view P99 at most 8 s (C8), over offered frontiers. P99 weights frontier observations; $A_{2,8}$ weights duration, so a P99 pass may coexist with a small uncredited tail.

Native latency uses each file/request's original origin. Initial loading and fixed warmup precede normal service; fault trials charge replacement and recomputation on the original clocks. Board energy spans the common trial origin-to-confirmed physical-model-drain window. Tables state their aggregation: totals, per-run P99 medians, or observed ranges. Energy is normalized by offered audio and compared within the same runtime and workload; native sequential-feedback and batched-core values are not directly comparable.

We separately count visible-frontier withdrawals, erased code points and deficit below the previously observed frontier. Retrospective whole-text support compares a first actual view with later selected text at that view's own frontier. This is distinct from dependency validity [SUP], human word/character error rate, and new character count. Independent checks reconstruct complete selected $F/Z/P/\mathrm{text}$ histories and actual consumed input paths from original outputs.

B. Does retaining computation retain delivered service?

Retaining computation alone does not preserve delivered progress. Figure 3 shows the preselected primary setting: Qwen, $\rho = 1$ (2-s feedback), and staggered arrivals. With the same selected histories, computation retention raises covered audio from 654.115 to 664.150 s but withdrawals from 12 to 17. View retention alone reduces withdrawals to 1 but covers 628.435 s. RevTrace jointly covers 680.760 s with zero frontier withdrawals: 26.645 s above recomputation and 16.610 s above computation-only. The retained computation can produce an ahead view, but a snapshot Consumer still replaces it when selected feedback installs. The relative-progress analysis below checks those actual compatible applications.

Cohort-wise credit interaction, $I = Y_{11} - Y_{10} - Y_{01} + Y_{00}$, is $+18.530$, $+6.205$, and $+17.555$ s. These are three queue-coupled cohort contrasts, not independent deadline windows. RevTrace's text remains revisable: 569 characters are erased despite zero frontier withdrawal. Median per-run display P99 is 2.329 s, above D2, so the benefit is retained progress under overload rather than a whole-run service pass.

The alternative joint policy also yields zero withdrawals and 569 erased characters, with 667.530 s coverage and 2.534-s median P99. It reproduces the supported-view behavior on the common substrate. Across confirmation, 144 matched history groups and 22 positive-control pairs agree on complete selected histories. Thus the comparison holds the feedback task fixed while changing how valid work becomes service.

Finding 1: retained computation does not ensure retained service. [EL] can retain work whose presentation a snapshot Consumer still removes. Joint [EL]/[SUP] coordination preserves the supported ahead view.

C. Timely native service with original full calls

Input-confirmed full calls improve timely service while preserving the original feedback task. On eight previously unused speakers, RevTrace raises audio coverage meeting both deadlines from 40.6% to 93.9% and whole-run passes from zero to ten of sixteen (Table III). A paired speaker bootstrap, keeping both timing repeats together, gives a duration-weighted gain of 53.3 percentage points (95% CI [44.8, 59.6]; 20,000 resamples). All eight source-paired gains are positive, ranging from 27.8 to 61.4 percentage points. The StreamCorrect adaptation retains original learned correction and feedback, using fixed 500-ms inputs, no VAD, and FP32 execution [1]. Both author-built policies have two complete workers, the same ASR early-view Consumer and neural warmup. Conservative execution waits for selected parent feedback; RevTrace starts conditional full calls and retains them when their complete inputs are confirmed. This tests complete-call overlap against waiting for selected parent feedback; core factorization separately tests computation/view retention.

a) Preserving the original selected task: Metadata selection froze eight unused AISHELL-1 test speakers [14] before reference or service inference, with one complete 8–9-s utterance per speaker. Two policies and two fresh repeats give 32 runs with balanced policy order and GPU roles swapped across repeats. Reference checks and independent actual-Consumer audits bind all 564 selected transitions to complete original inputs/outputs, correction and returned IDs. Final quality is identical across policies and repeats: 23 edits over 174 unique reference characters (13.22% CER). Reference generation is charged separately from service.

TABLE III
COMPLETE-CALL RETENTION ON EIGHT PREVIOUSLY UNUSED SPEAKERS. EACH POLICY OFFERS 137.864 AUDIO-S, WITH TWO FRESH TRIALS PER SPEAKER. COVERAGE IS A TOTAL; D99 IS THE MEDIAN PER-RUN DISPLAY P99. BOTH-BOARD ENERGY IS NORMALIZED BY OFFERED AUDIO.

Policy	Timely coverage (s)	D99 (s)	Energy (J/audio-s)	Pass
Conservative	56.000	5.027	690.989	0/16
RevTrace	**129.460**	**1.762**	654.845	**10/16**

b) Energy over the complete service window: On the new speakers, RevTrace uses 5.23% less common-window board energy than Conservative. The origin-to-physical-drain window includes issued work that is not used.

c) Where reusable lookahead runs out: The six failed trials come from S0901, S0902 and S0915. S0915 contributes 6.996 of the 8.404 uncredited audio-s: both repeats miss the same seven frontiers from 6 s through EOS. Repeated original corrections refute the next inputs. Each repeat's six late non-EOS replacements dispatch 2–5 ms after the recorded parent-install event and spend 0.53–0.74 s in original inference per 0.5-s input. At the 6-s frontier in repeat 0, dispatch is at 7.591 s, the ASR boundary at 7.976 s, and actual consumption at 8.023 s crosses D2 at 8.000 s. S0901 retains a matching early EOS call after slow preceding recognition; its ASR boundary still arrives late. S0902 loses lookahead after full-input refutations. In S0915, repeated refutations exhaust reusable lookahead as the original inference path outgrows the input cadence.

d) More selected content meets the deadlines: Figure 4 separates deadline coverage from content support. On the eight new speakers, timely coverage whose whole first view retrospectively matches selected text at that view's own frontier G rises from 47.000 to 107.162 s (34.1% to 77.7% of all offered audio). No timely coverage has unknown agreement; raw and whitespace-stripped comparisons agree. More matching views meet the deadlines under the unchanged selected task. In the four-speaker resident bank, matching timely coverage rises from 24.0 to 50.259 s.

e) Benefits persist across resident requests: RevTrace raises joint D2/C8 passes from 0/8 to 7/8 requests, and timely coverage from 31.000 to 64.119 of 66.340 s. Models load once per trial; both worker PIDs persist across four pre-registered independent requests. The next request starts only after actual certified consumption of the previous file EOS. Two policies and two fresh trials preserve all 272 original transitions and each request's own input clock. The second S0908 repeat is the only failing request, with D99 2.229 s. This single-outstanding closed-loop contract tests residency.

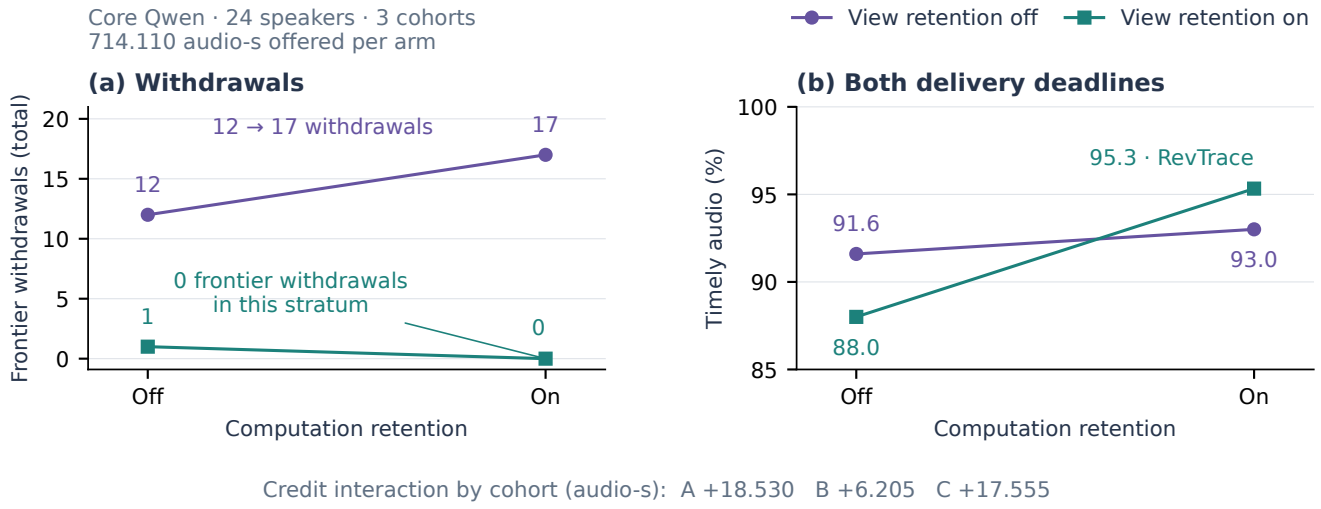


Fig. 3. Factorial interactions in the preselected Qwen, $\rho = 1$, staggered setting. Points pool three eight-speaker cohorts; each arm offers 714.110 s. (a) Total frontier withdrawals. (b) Offered-audio fraction meeting display 2 s and certified 8 s deadlines. Joint retention is RevTrace. All four arms and the alternative joint positive control fail whole-run D2 (15 runs).

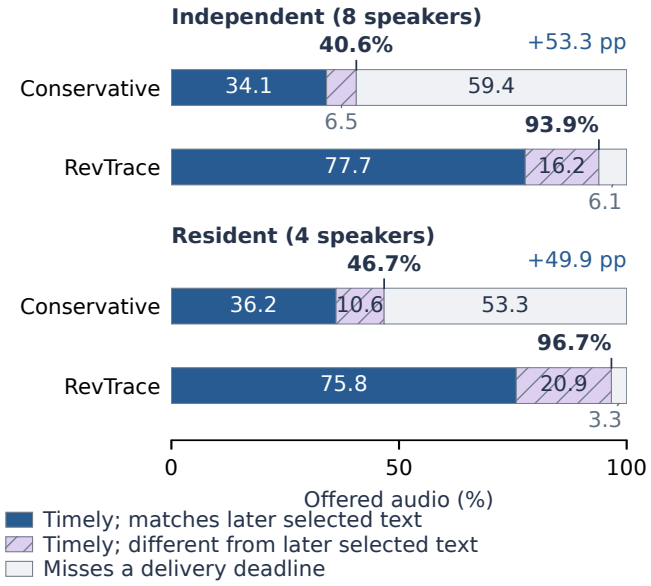


Fig. 4. More matching content meets both deadlines: display within 2 s and certified coverage within 8 s. Independent eight-speaker and resident four-speaker cohorts offer 137.864 and 66.340 audio-s per policy. Stacks partition timely coverage matching later selected whole text at its own frontier, other timely coverage, and missed deadlines. Retrospective agreement is distinct from online [SUP] and human accuracy. Boundary labels sum both timely categories, in rounded percentages.

f) Benefits are not confined to the D2 cutoff: In the existing four-speaker banks, we recompute offered-audio coverage over display budgets from 0.5 to 6 s, keeping every original frontier and EOS tail. The confirmation budget remains C8. At 1.5 s, coverage rises from 36.2% to 84.0% in matched runs and from 36.2% to 78.7% in resident requests; both policies reach 100% at sufficiently loose budgets. At D2, all four speaker-paired gains are positive in each bank. The advantage spans tighter delivery budgets and converges as time pressure eases; the original D2/C8 pass rule remains unchanged.

g) Input confirmation and view retention: An earlier 16-run confirmation bank on the four existing sources has 92 promoted calls whose recognition begins before selected parent completion; 20 predict successor states that differ from their original selected results. These observations separate input eligibility from prediction correctness.

The service gain comes from earlier execution of complete calls whose inputs are later confirmed. A resident execution separately exercises view retention: a 5.0-s whole view survives a compatible 4.5-s selected update with the same tip and job. Its 0.817-s Consumer event-state holding interval spans that update and ends at legitimate supersession. Each existing four-speaker RevTrace bank also contains two necessary incompatible withdrawals.

Finding 2: confirmed inputs accelerate the original task. Original selection [AU] remains intact when [EL] confirms early full calls; whole-view retention still requires [SUP] independently.

D. Observed protocol-path costs

We replay four actual source/policy traces in 12 fresh CPU processes and run 16 normal native trials: two fixed sources, two policies, recording off/on and two fresh repeats. Replays reproduce the complete fact-journal bytes and Consumer semantics. All eight off/on pairs preserve the selected states, original corrections and returned IDs.

Table IV reports the eight trials with timing enabled. Synchronous storage accounts for most durable-commit wall time. Service comparisons include these complete-state validation, persistence and whole-view processing costs.

E. When does the opportunity appear or disappear?

Relative progress creates the opportunity: a supported whole view is ahead when a selected update installs. In the primary

TABLE IV
PROTOCOL-PATH WALL TIMES: RANGES OF PER-RUN MEANS IN EIGHT TIMING-ENABLED NATIVE TRIALS. COMMIT INCLUDES VALIDATION AND NESTED FSYNC; CONSUMER APPLICATION INCLUDES FACT/VIEW CHECKS AND SYNCHRONOUS LOGGING.

Operation	Wall time (ms/call)
Durable full-state commit	20.35–27.25
Owner fsync	15.78–22.18
Whole-view Consumer application	18.58–23.91

TABLE V
OPERATING WORKLOADS: FOUR ARMS AND THREE FRESH PROCESSES PER ROW. PASS COVERS ALL 12 RUNS; D99 SPANS THREE REVTRACE RUNS. TIMELY-LOSS WINDOWS LIST COMPUTATION-ONLY/REVTRACE COUNTS BY REPEAT.

Workload	D2/C8 pass	RevTrace D99 (s)	Timely-loss windows
8 sync	12/12	1.426–1.528	1,0,0 /0,0,0
8 staggered	12/12	1.704–1.955	1,2,1 /0,0,0
12 staggered	0/12	2.197–2.299	1,1,3 /0,0,0
16 staggered	0/12	2.801–3.223	10,12,9 /0,0,0

RevTrace runs, 18 of 368 updates encounter and retain a supported ahead view; computation-only encounters 17 and clears all 17. Under synchronized arrivals, the opportunity largely disappears: RevTrace encounters only 3 of 368 and covers 0.790 s less than computation-only. A fresh staggered repeat on cohort A also changes the timing ranking: 226.320 versus 230.565 s, $I = -2.245$ s. Compatible relative progress determines the benefit; the three positive primary contrasts do not establish an invariant speed advantage.

Table V summarizes 36 predeclared development runs (8 synchronized, 8 staggered, 16 staggered sessions) and 12 exploratory midpoint runs (12 staggered). Each arm offers 121.205/185.300/250.180 s at 8/12/16 sessions. These are explicit workloads with different source mixes. At 8 sessions all arms meet whole-run D2/C8; RevTrace synchronized display P99 is 1.426–1.528 s and staggered reaches 1.955 s. Two 0.215-s tails remain uncredited despite their P99 passes. At 12 and 16 sessions all runs fail D2 and pass C8. At 16, RevTrace's extra coverage over computation-only is +18.910,+5.055,−7.225 s; P99 improves in one repeat. RevTrace's continuity distinction is stable: compatible withdrawals are zero versus 16/17/12.

A secondary analysis follows intervals already delivered within D2 whose eventual certified coverage also meets C8. At each selected application, the complete dependency path classifies compatible loss separately from necessary revision. Loss ends when coverage returns. If it has not returned, observation stops at the next selected root without treating that boundary as recovery. Computation-only has 10/12/9 such windows at 16 sessions, with 24.903/23.635/15.736 audio-s×wall-s deficit; RevTrace has none. Holes intersect the original D2 read point for 6.000/10.675/4.000 s of distinct audio.

Finding 3: continuity can improve without meeting the whole-run deadline. Joint retention removes the measured compatible withdrawals, while all 12- and 16-session runs still miss D2.

a) Quality and cost: The primary Qwen arms share WER 26/1908 (1.36%) and selected histories. Retrospective same-frontier whole-text agreement leaves their credits unchanged. For RevTrace outside the primary setting, requiring retrospective raw-text agreement excludes 2 s in the 16-session third repeat and 2 s in each midpoint's first two repeats, although the frozen normalizer agrees. In the primary setting, board energy spans 59.16–59.73 J/audio-s across five arm medians. At 16 sessions, RevTrace reduces total model time by 0.66–1.59 s over recomputation, while wall-time and energy differences change sign. Retained progress is the measured benefit; inference cost and overload are independent boundaries.

F. Preserving acquired results through process failures

The final coordinator preserves acquired results without granting old messages new authority. Both policies share models, durability, guards, recovery and Consumer. Initial loading and fixed warmup precede service; replacement loading, replay, recomputation and physical drain retain the original clock.

Six conditions, two policies and two fresh processes on one development clip give 24 short trials, plus four EOS/compatible-view regressions. Together with eight sustained-state trials below, these 36 trials contain 3,976 durable transitions independently bound to complete original call receipts and actual Consumer applications, with zero illegal installations or incorrect owned-message rejections. Eight duplicate/late messages cause no selected-state mutation; eight worker deaths have confirmed physical replacements. Conditional calls supply 354 durable facts: 352 have original Owner installation records, and two first reach the Consumer through recovery. In four worker-after-ASR cases, a previously consumed whole view remains across producer exit/fencing until legitimate correction. Its support, rather than the producer's continued presence, determines retention.

Worker and Owner failures preserve legality but have different service costs (Table VI). Normal and duplicate RevTrace each pass one of two trials; the other misses D2 only at EOS. All worker-fault trials fail D2 but pass C8; Owner-fault trials fail both. Old durable facts reappear in 1.36–1.81 s, whereas new certified post-fault completions take 64.61–108.08 s in Owner cases. A surviving worker can finish a call before replacement readiness, so replay, new results and replacement loading have separate recovery clocks.

a) Complete buffers across producer death: Two matching and two mismatching timing controls isolate a complete result already registered by the Owner. The fixture holds the parent after original ASR and the child after sending its complete original result. We terminate the producer, verify exit, CUDA release and sealed-channel validation, fence new messages, and release original parent correction. Both matching results install and are consumed once with their original job, birth and incarnation; both mismatches are rejected against the actual same-frontier parent. Each positive case also completes a

TABLE VI

LEGALITY SURVIVES WORKER AND OWNER FAULTS. EACH CONDITION/POLICY COVERS TWO FRESH TRIALS ON THE SAME CLIP; ALL 24 TRIALS HAVE ZERO ILLEGAL INSTALLATIONS. REJECT/ΔC COUNTS REJECTED MESSAGES/STATE MUTATIONS. FAULT-TO-NEW-FACT RUNS FROM THE FAULT TO ACTUAL CERTIFIED CONSUMER CONSUMPTION OF A POST-FAULT COMPLETION. PASS COUNTS JOINT D2/C8; EOS COUNTS D2 MISSES. RANGES ARE MINIMA/MAXIMA.

Condition	Policy	Reject / ΔC	Fault-to-new-fact (s)	D2/C8 pass	EOS miss	Energy (J)
No fault	Conservative	0 / 0	–	0/2	2	6566–6574
	RevTrace	0 / 0	–	1/2	1	5896–5911
Duplicate / late	Conservative	4 / 0	–	0/2	2	5701–5893
	RevTrace	4 / 0	–	1/2	1	5538–5846
Worker after ASR	Conservative	0 / 0	1.90–2.22	0/2	2	5182–5401
	RevTrace	0 / 0	2.15–2.21	0/2	2	5447–5647
Worker partial frame	Conservative	0 / 0	2.10–2.25	0/2	2	4775–5178
	RevTrace	0 / 0	1.76–2.23	0/2	2	5492–5594
Owner before durable	Conservative	0 / 0	86.42–108.08	0/2	2	24549–28019
	RevTrace	0 / 0	64.61–107.09	0/2	2	19571–26557
Owner after durable	Conservative	0 / 0	83.90–87.05	0/2	2	22415–24200
	RevTrace	0 / 0	101.80–104.44	0/2	2	26483–27171

subsequent independent request. These controlled interleavings establish result retention and rejection, rather than fault-time SLOs.

Finding 4: producer failure and result invalidity are different. Producer exit revokes future message authority. An acquired result retains its original identity and is checked against its actual parent.

b) Sustained-state stress and its limits: Two metadata-selected AISHELL-1 speakers supply 24 utterances each, concatenated as exact PCM into 228.510-s and 208.100-s streams without resets. A continuous original-engine reference and 875 restored steps agree on complete state, correction and IDs across 398/357 buffer shifts and 353/252 context changes. Eight serving trials produce 3,500 matching selected steps. Unpinned terminal and audit windows stay at most 64 entries. This limit does not bound live dependencies, state/output bytes, RSS or cumulative logs, which are measured separately.

All eight trials fail D2/C8 and reproduce severe original-task insertion errors, with higher energy for RevTrace. They validate state capture and collection through long executions, rather than sustained ASR service; the artifact reports the complete latency, quality and resource metrics. Across all 36 trials, the checker preserves 90 compatible-ahead dependency-tip applications and records 1,118 necessary incompatible withdrawals. These counts concern dependency tips, rather than new text.

Input confirmation and use have different denominators. The dispatch audit confirms 40/57 conditional inputs on normal short traces, 58/90 under controlled faults, and 96/924 and 166/830 on the two no-reset sources; none remain unresolved. Six confirmed jobs do not become durable under injected failures. The fault trials supply 52 durable, consumed results: 50 have original-Owner installation records and two first reach the Consumer through recovery.

G. From physical release to supported service

A separate 24-run core study crosses R and K with S enabled, two arrival patterns and three fresh processes, using a frozen wrong-conditioning fixture. Selected histories and WER $14/687$ match; all 675 same-input/prompt/audio/peer-order pairs return identical outputs. In synchronized runs, computation retention releases the next 16-row computation 473–577 ms earlier. All 96 following rows are actually consumed and their whole texts match later same-frontier Canonical text. Only 6/6/1 rows switch from deadline miss to hit. Enabling covered exit (K) changes credit by +7.970 to +15.255 s when synchronized and -26.000 to $+21.780$ s when staggered, without consistent energy savings. Request acknowledgment still precedes physical drain. This is a measured release-to-service opportunity, with no stable positive $R \times K$ interaction.

TABLE VII

REPRESENTATIVE REUSE OBJECTS AND REVTRACE'S FEEDBACK-SPECIFIC OBLIGATION. ENTRIES DESCRIBE THE COMPARED MECHANISMS' FOCUS.

Family	Reuse object	Feedback obligation here
IU / Adapton	Units / dependency computations	Bind complete feedback inputs and owned calls.
SSBD / PredGen	Draft tokens / candidate replies	Preserve original recursive correction and update.
StreamCorrect	Selected correction prefix	Overlap full calls and validate the actual whole view.
Retire	Contract-valid successor state	Derive input and view actions from selected neural feedback.

VI. RELATED WORK

a) Incremental information and dependency reuse: The incremental-unit model separates newest committed information from the active right edge and propagates dependency revocation [2]. Its commit promises that a unit will not be revoked. Here, certification records acceptance by the protected selection protocol; later selected facts may still revise the transcript. Adapton checks recorded dependencies; Nominal Adapton combines names with arguments and dependency checks to recover reusable work [3], [4]. Differential Dataflow maintains incremental computations across evolving versions [15]. These foundations for dependency-aware reuse (Table VII) support

RevTrace's complete-state call and Consumer witnesses at independently selected root applications. Our sufficient witness follows recorded executions; it does not transfer a from-scratch theorem to independently repeated floating-point inference.

b) Revisable speech services and speculation: Whisper-Streaming converts an offline recognizer into incremental service using LocalAgreement [16]. StreamCorrect adds learned correction and feeds selected corrections into subsequent recognition [1]. RevTrace's native adapter retains this correction/feedback task while coordinating asynchronous full calls and whole-view consumption.

SSBD coordinates old-output draft reuse with display-only suffix masking [5]. It revalidates a draft under updated input and includes an unbiased configuration. RevTrace follows complete calls that actually executed to establish their eligibility under newly selected feedback state, then checks the dependency witness for the Consumer's whole current view. PredGen provides a close precedent for speculation while speech input evolves across recognition, language generation, and speech synthesis [6]. Our comparison holds the complete selected feedback task fixed across the retention policies. RelayS2S uses a learned verifier to commit a fast-path prefix that constrains subsequent slow-path generation [17]. Our native calls instead confirm their complete inputs and retain the original correction and feedback update. Work on ASR alignment drift also establishes the importance and cost of audio position state [18]; our contract includes that state rather than treating text equality as sufficient.

c) Inference reuse and verification: Orca addresses iteration-level transformer scheduling [19]. vLLM's PagedAttention shares KV blocks within and across requests, while SGLang's RadixAttention reuses token-prefix KV state across program calls [20], [21]. SGLang's API speculation retains generation beyond a stop condition and matches it to later program primitives. Speculative decoding verifies draft tokens against a target model while preserving its output distribution [22]. RevTrace binds each reused full call to its eventual selected parent, retains original correction/update, and separately validates the Consumer's current whole view.

d) Execution, authorization, and recovery: Retire enforces runtime-supplied revocation scopes and reuse contracts, certifying state inheritance while reclaiming old resources [23]. RevTrace derives complete-call eligibility and whole-view actions from an unchanged selected feedback task: an unsupported view can be withdrawn while a future call's input remains unresolved, and confirming that input does not select the call's predicted result. Our covered-exit policy is execution support, without a new general cancellation claim. The parent-version control does not reproduce the complete Retire system.

Optimistic concurrency control validates concurrent work before commit [24]; multiversion control manages reads across data versions [25], while snapshot isolation supplies a distinct transaction guarantee [26]. Write-ahead recovery provides durable replay [9], while distributed snapshots capture process/channel state [27]. Stream processors restore state through lineage or coordinated snapshots [28], [29]. RevTrace replays a locally verified selected prefix and releases physical ownership at terminal/drain, unlike time-bounded cache leases [30]. These foundations supply ordering and recovery; the feedback adapter adds complete-input eligibility and whole-view retention.

VII. Discussion

RevTrace coordinates permission to reuse a computation with support for the view currently delivered. This keeps a trusted revision from unnecessarily discarding work that still has a valid use.

a) Evidence and comparison scope: Native confirmation uses one 8–9-s utterance per speaker; timing repeats do not add acoustic sources. Native and core policy comparisons isolate their respective retention effects, rather than rank external speculative-serving systems.

b) Transferring the contract: An adapter must (1) capture and restore all state affecting later inference, selection or feedback; (2) bind complete original results, selection/update, identities and durable transitions; and (3) register the actual Consumer's whole-view provenance and dependencies for retention or withdrawal. Section II instantiates these obligations for Qwen feedback tokens and native buffer/context/alignment state. Hidden caches, external state or irreversible effects require additional adapter support. Retention still pays synchronization, queueing and inference; resource admission governs when to launch more work [31]. Cold Owner recovery uses the surviving components of Section II-A.

c) Reproducibility: The anonymous companion rebuilds summaries, paired confidence intervals and figures from bounded records. It includes source snapshots, frozen configurations, per-run identities and endpoint-checking scripts. Full-journal rechecking and model reruns additionally require the indexed journals, recordings, weights and dependencies; a hash index alone does not recheck their content.

VIII. Conclusion

Trusted feedback changes which uses of speculative work remain valid. RevTrace checks complete-call eligibility, original selection authority and actual whole-view support separately. Under the declared contract, these witnesses preserve selected-task legality and prevent avoidable withdrawals at compatible applications. Factorial controls show why retaining computation alone is insufficient; independent native trials turn confirmed full calls into timely service without changing original correction or feedback. The practical lesson is to bind each retained use to its own evidence, including received results whose producer has failed.

AI Tool Usage

ChatGPT/Codex assisted literature discovery, manuscript drafting/revision and the refinement argument. It also assisted runtime extensions, experiment orchestration, and validation/analysis code for figures and tables. The authors retain full responsibility.

References

[1] T. D. Vu, Z. Xie, W. Zhao, V. J. Wei, D. Jiang, and Y. R. Fung, "StreamCorrect: Self-supervised error correction for real-time ASR with offline recognizers," in *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, Oct. 2026, pp. 2391–2403. [Online]. Available: https://aclanthology.org/2026.emnlp-industry.142/

[2] D. Schlangen and G. Skantze, "A general, abstract model of incremental dialogue processing," *Dialogue & Discourse*, vol. 2, no. 1, pp. 83–111, 2011. [Online]. Available: https://aclanthology.org/2011.dnd-2.11/

[3] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, demand-driven incremental computation," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2014, pp. 156–166. [Online]. Available: https://matthewhammer.org/adapton/adapton-pldi2014.pdf

[4] M. A. Hammer, J. Dunfield, K. Headley, N. Labich, J. S. Foster, M. Hicks, and D. Van Horn, "Incremental computation with names," in *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 2015, pp. 748–766. [Online]. Available: https://arxiv.org/abs/1503.07792v2

[5] L. Zeng, H. Deng, K. Shu, and S. Wang, "Self-speculative biased decoding for faster re-translation," 2026. [Online]. Available: https://arxiv.org/abs/2509.21740v2

[6] S. Li and A. Grover, "PredGen: Accelerated inference of large language models through input-time speculation for real-time speech interaction," in *Second Conference on Language Modeling*, 2025. [Online]. Available: https://openreview.net/forum?id=2Kl8Ztw6wk

[7] F. B. Schneider, "Implementing fault-tolerant services using the state machine approach: A tutorial," *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, 1990.

[8] M. Abadi and L. Lamport, "The Existence of Refinement Mappings," *Theoretical Computer Science*, vol. 82, no. 2, pp. 253–284, 1991. [Online]. Available: https://www.sciencedirect.com/science/article/pii/030439759190224P

[9] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.

[10] A. D. Birrell and B. J. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, Feb. 1984. [Online]. Available: https://www.microsoft.com/en-us/research/publication/implementing-remote-procedure-calls/

[11] X. Shi, X. Wang, Z. Guo, Y. Wang, P. Zhang, X. Zhang, Z. Guo, H. Hao, Y. Xi, B. Yang, J. Xu, J. Zhou, and J. Lin, "Qwen3-ASR Technical Report," *arXiv preprint arXiv:2601.21337*, 2026.

[12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 28 492–28 518. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html

[13] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR Corpus Based on Public Domain Audio Books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.

[14] H. Bu, J. Du, X. Na, B. Wu, and H. Zheng, "AISHELL-1: An Open-Source Mandarin Speech Corpus and A Speech Recognition Baseline," in *2017 20th Conference of the Oriental Chapter of the International Coordinating Committee on Speech Databases and Speech I/O Systems and Assessment (O-COCOSDA)*, 2017, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/8384449/

[15] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential dataflow," in *Sixth Biennial Conference on Innovative Data Systems Research*, 2013. [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf

[16] D. Macháček, R. Dabre, and O. Bojar, "Turning Whisper into Real-Time Transcription System," in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*, 2023, pp. 17–24.

[17] L. Mai and J. Liang, "RelayS2S: Dual-path speculative generation for real-time dialogue," in *Findings of the Association for Computational Linguistics: EMNLP 2026*. Association for Computational Linguistics, Oct. 2026, pp. 25 379–25 396. [Online]. Available: https://aclanthology.org/2026.findings-emnlp.1201/

[18] X. Wang, H. Zhou, Z. Zhao, S. Meng, K. Bai, D. Shen, X.-W. Chang, and A. Smola, "Alignment drift in single-model speculative decoding for ASR: Mechanism, correction, and cost," 2026. [Online]. Available: https://arxiv.org/abs/2608.12703v1

[19] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2022, pp. 521–538.

[20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626. [Online]. Available: https://arxiv.org/abs/2309.06180

[21] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html

[22] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19 274–19 286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[23] Y. Zhang, R. Sharma, N. Vegesna, Z. Fu, C. Liu, and T. Krishnamurthy, "Serving a revisable world: Versioned execution for interruptible agents," 2026. [Online]. Available: https://arxiv.org/abs/2610.01160v1

[24] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[25] P. A. Bernstein and N. Goodman, "Multiversion concurrency control—theory and algorithms," *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, 1983.

[26] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. ACM, 1995, pp. 1–10.

[27] K. M. Chandy and L. Lamport, "Distributed snapshots: Determining global states of distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985.

[28] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. ACM, 2013, pp. 423–438.

[29] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State Management in Apache Flink: Consistent Stateful Distributed Stream Processing," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1718–1729, 2017. [Online]. Available: https://www.vldb.org/pvldb/vol10/p1718-carbone.pdf

[30] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles*. ACM, 1989, pp. 202–210.

[31] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable internet services," in *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles*. ACM, 2001, pp. 230–243.